

XGBoost Parameters

Before running XGBoost, we must set three types of parameters: general parameters, booster parameters and task parameters.

- **General parameters** relate to which booster we are using to do boosting, commonly tree or linear model
- **Booster parameters** depend on which booster you have chosen
- **Learning task parameters** decide on the learning scenario. For example, regression tasks may use different parameters with ranking tasks.

! Note

Parameters in R package

In R-package, you can use `.` (dot) to replace underscore in the parameters, for example, you can use `max.depth` to indicate `max_depth`. The underscore parameters are also valid in R.

- [Global Configuration](#)
- [General Parameters](#)
 - [Parameters for Tree Booster](#)
 - [Parameters for Non-Exact Tree Methods](#)
 - [Parameters for Categorical Feature](#)
 - [Additional dropout parameters for tree boosters](#)
 - [Parameters for Linear Booster](#) (`booster=gblinear`)
- [Learning Task Parameters](#)
 - [Parameters for Tweedie Regression](#) (`objective=reg:tweedie`)
 - [Parameter for using Pseudo-Huber](#) (`reg:pseudohubererror`)
 - [Parameter for using Quantile Loss](#) (`reg:quantileerror`)
 - [Parameter for using Expectile Loss](#) (`reg:expectileerror`)
 - [Parameter for using AFT Survival Loss](#) (`survival:aft`) and Negative Log Likelihood of AFT metric (`aft-nloglik`)
 - [Parameters for learning to rank](#) (`rank:ndcg`, `rank:map`, `rank:pairwise`)

Global Configuration

The following parameters can be set in the global scope, using `xgboost.config_context()` (Python) or `xgb.set.config()` (R).

- `verbosity`: Verbosity of printing messages. Valid values of 0 (silent), 1 (warning), 2 (info), and 3 (debug).
- `use_rmm`: Whether to use RAPIDS Memory Manager (RMM) to allocate cache GPU memory. The primary memory is always allocated on the RMM pool when XGBoost is built (compiled) with the RMM plugin enabled. Valid values are `true` and `false`. See [Using XGBoost with RAPIDS Memory Manager \(RMM\) plugin](#) for details.
- `use_cuda_async_pool` [default=false]

Whether to use the device memory pool in the CUDA driver. This option is not available if XGBoost is built with RMM support, as it is the same as using the RMM `CudaAsyncMemoryResource` pool.

Added in version 3.2.0.

⚠ Warning

This is an experimental feature and is subject to change without notice. Windows is not supported yet.

- `nthread`: Set the global number of threads for OpenMP. Use this only when you need to override some OpenMP-related environment variables like `OMP_NUM_THREADS`. Otherwise, the `nthread` parameter from the Booster and the DMatrix should be preferred as the former sets the global variable and might cause conflicts with other libraries.

General Parameters

- `booster` [default= `gbtree`]
 - Which booster to use. Can be `gbtree`, `gblinear` or `dart`; `gbtree` and `dart` use tree based models while `gblinear` uses linear functions.
 - Dropout parameters like `rate_drop` can be used directly with tree models. `booster=dart` remains supported for compatibility.

Deprecated since version 3.3.0: `booster=gblinear` is deprecated and support will be removed in a future release.

- Device for XGBoost to run. User can set it to one of the following values:

- `cpu`: Use CPU.
- `cuda`: Use a GPU (CUDA device).
- `cuda:<ordinal>`: `<ordinal>` is an integer that specifies the ordinal of the GPU (which GPU do you want to use if you have more than one devices).
- `gpu`: Default GPU device selection from the list of available and supported devices. Only `cuda` devices are supported currently.
- `gpu:<ordinal>`: Default GPU device selection from the list of available and supported devices. Only `cuda` devices are supported currently.

For more information about GPU acceleration, see [XGBoost GPU Support](#). In distributed environments, ordinal selection is handled by distributed frameworks instead of XGBoost. As a result, using `cuda:<ordinal>` will result in an error. Use `cuda` instead.

- `verbosity` [default=1]
 - Verbosity of printing messages. Valid values are 0 (silent), 1 (warning), 2 (info), 3 (debug). Sometimes XGBoost tries to change configurations based on heuristics, which is displayed as warning message. If there's unexpected behaviour, please try to increase value of verbosity.
- `validate_parameters` [default to `false`, except for Python, R and CLI interface]
 - When set to True, XGBoost will perform validation of input parameters to check whether a parameter is used or not. A warning is emitted when there's unknown parameter.
- `nthread` [default to maximum number of threads available if not set]
 - Number of parallel threads used to run XGBoost. When choosing it, please keep thread contention and hyperthreading in mind.
- `disable_default_eval_metric` [default= `false`]
 - Flag to disable default metric. Set to 1 or `true` to disable.

Parameters for Tree Booster

- `eta` [default=0.3, alias: `learning_rate`]
 - Step size shrinkage used in update to prevent overfitting. After each boosting step, we can directly get the weights of new features, and `eta` shrinks the feature weights to make the boosting process more conservative.
 - range: [0,1]
- `gamma` [default=0, alias: `min_split_loss`]

- Minimum loss reduction required to make a further partition on a leaf node of the tree. The larger `gamma` is, the more conservative the algorithm will be. Note that a tree where no splits were made might still contain a single terminal node with a non-zero score. This is the same γ described in the [Introduction to Boosted Trees](#).
- range: $[0, \infty]$
- `max_depth` [default=6, type=int32]
 - Maximum depth of a tree. Increasing this value will make the model more complex and more likely to overfit. 0 indicates no limit on depth. Beware that XGBoost aggressively consumes memory when training a deep tree. `exact` tree method requires non-zero value.
 - range: $[0, \infty]$
- `min_child_weight` [default=1]
 - Minimum sum of instance weight (hessian) needed in a child. If the tree partition step results in a leaf node with the sum of instance weight less than `min_child_weight`, then the building process will give up further partitioning. In linear regression task, this simply corresponds to minimum number of instances needed to be in each node. The larger `min_child_weight` is, the more conservative the algorithm will be.
 - With vector leaf, the mean Hessian across targets is used to compare against the `min_child_weight`.
 - range: $[0, \infty]$
- `max_delta_step` [default=0]
 - Maximum delta step we allow each leaf output to be. If the value is set to 0, it means there is no constraint. If it is set to a positive value, it can help making the update step more conservative. Usually this parameter is not needed, but it might help in logistic regression when class is extremely imbalanced. Set it to value of 1-10 might help control the update.
 - range: $[0, \infty]$
- `subsample` [default=1]
 - Subsample ratio of the training instances. Setting it to 0.5 means that XGBoost would randomly sample half of the training data prior to growing trees. and this will prevent overfitting. Subsampling will occur once in every boosting iteration.
 - range: $(0, 1]$
- `sampling_method` [default= `uniform`]

Changed in version 3.2.0:

XGBoost supports both CPU and GPU for gradient-based sampling.

- The method to use to sample the training instances.

- `gradient_based`: the selection probability for each training instance is proportional to the *regularized absolute value* of gradients (more specifically, $\sqrt{g^2 + \lambda h^2}$). `subsample` may be set to as low as 0.1 without loss of model accuracy. Note that this sampling method is only supported when `tree_method` is set to `hist`; other tree methods only support `uniform` sampling.

! Note

When working with reduced gradient for multi-target models, the accuracy of gradient-based sampling might be sub-optimal. The sampling is performed using the split gradient, which may not be optimal with the full gradient. Use uniform sampling as an alternative.

- `colsample_bytree`, `colsample_bylevel`, `colsample_bynode` [default=1]
 - This is a family of parameters for subsampling of columns.
 - All `colsample_by*` parameters have a range of (0, 1], the default value of 1, and specify the fraction of columns to be subsampled.
 - `colsample_bytree` is the subsample ratio of columns when constructing each tree. Subsampling occurs once for every tree constructed.
 - `colsample_bylevel` is the subsample ratio of columns for each level. Subsampling occurs once for every new depth level reached in a tree. Columns are subsampled from the set of columns chosen for the current tree.
 - `colsample_bynode` is the subsample ratio of columns for each node (split). Subsampling occurs once every time a new split is evaluated. Columns are subsampled from the set of columns chosen for the current level. This is not supported by the exact tree method.
 - `colsample_by*` parameters work cumulatively. For instance, the combination `{'colsample_bytree':0.5, 'colsample_bylevel':0.5, 'colsample_bynode':0.5}` with 64 features will leave 8 features to choose from at each split.

Using the Python or the R package, one can set the `feature_weights` for DMatrix to define the probability of each feature being selected when using column sampling. There's a similar parameter for `fit` method in sklearn interface.

- `lambda` [default=1, alias: `reg_lambda`]
 - L2 regularization term on weights. Increasing this value will make model more conservative. This is the λ described in the [Introduction to Boosted Trees](#).
 - range: $[0, \infty]$
- `alpha` [default=0, alias: `reg_alpha`]
 - L1 regularization term on weights. Increasing this value will make model

- `tree_method` string [default= `auto`]
 - The tree construction algorithm used in XGBoost. See description in the [reference paper](#) and [Tree Methods](#).
 - Choices: `auto` , `exact` , `approx` , `hist` , this is a combination of commonly used updaters. For other updaters like `refresh` , set the parameter `updater` directly.
 - `auto` : Same as the `hist` tree method.
 - `exact` : Exact greedy algorithm. Enumerates all split candidates.
 - `approx` : Approximate greedy algorithm using quantile sketch and gradient histogram.
 - `hist` : Faster histogram optimized approximate greedy algorithm.
- `scale_pos_weight` [default=1]
 - Control the balance of positive and negative weights, useful for unbalanced classes. A typical value to consider: `sum(negative instances) / sum(positive instances)` . See [Parameters Tuning](#) for more discussion. Also, see Higgs Kaggle competition demo for examples: [R](#), [py1](#), [py2](#), [py3](#).
- `updater`
 - A comma separated string defining the sequence of tree updaters to run, providing a modular way to construct and to modify the trees. This is an advanced parameter that is usually set automatically, depending on some other parameters. However, it could be also set explicitly by a user. The following updaters exist:
 - `grow_colmaker` : non-distributed column-based construction of trees.
 - `grow_histmaker` : distributed tree construction with row-based data splitting based on global proposal of histogram counting.
 - `grow_quantile_histmaker` : Grow tree using quantized histogram.
 - `grow_gpu_hist` : Enabled when `tree_method` is set to `hist` along with `device=cuda` .
 - `grow_gpu_approx` : Enabled when `tree_method` is set to `approx` along with `device=cuda` .
 - `refresh` : refreshes tree's statistics and/or leaf values based on the current data. Note that no random subsampling of data rows is performed.
 - `prune` : prunes the splits where `loss < min_split_loss` (or `gamma`) and nodes that have depth greater than `max_depth` .
- `refresh_leaf` [default=1]
 - This is a parameter of the `refresh` updater. When this flag is 1, tree leafs as well as tree nodes' stats are updated. When it is 0, only node stats are updated.
- `process_type` [default= `default`]
 - A type of boosting process to run.

- `update`: Starts from an existing model and only updates its trees. In each boosting iteration, a tree from the initial model is taken, a specified sequence of updaters is run for that tree, and a modified tree is added to the new model. The new model would have either the same or smaller number of trees, depending on the number of boosting iterations performed. Currently, the following built-in updaters could be meaningfully used with this process type: `refresh`, `prune`. With `process_type=update`, one cannot use updaters that create new trees.
- `grow_policy` [default= `depthwise`]
 - Controls a way new nodes are added to the tree.
 - Currently supported only if `tree_method` is set to `hist` or `approx`.
 - Choices: `depthwise`, `lossguide`
 - `depthwise`: split at nodes closest to the root.
 - `lossguide`: split at nodes with highest loss change.
- `max_leaves` [default=0, type=int32]
 - Maximum number of nodes to be added. Not used by `exact` tree method.
- `max_bin`, [default=256, type=int32]
 - Only used if `tree_method` is set to `hist` or `approx`.
 - Maximum number of discrete bins to bucket continuous features.
 - Increasing this number improves the optimality of splits at the cost of higher computation time.
- `num_parallel_tree`, [default=1]
 - Number of parallel trees constructed during each iteration. This option is used to support boosted random forest.
- `monotone_constraints`
 - Constraint of variable monotonicity. See [Monotonic Constraints](#) for more information.
- `interaction_constraints`
 - Constraints for interaction representing permitted interactions. The constraints must be specified in the form of a nest list, e.g. `[[0, 1], [2, 3, 4]]`, where each inner list is a group of indices of features that are allowed to interact with each other. See [Feature Interaction Constraints](#) for more information.
- `multi_strategy`, [default = `one_output_per_tree`]

Added in version 2.0.0.

This parameter is working-in-progress.

- The strategy used for training multi-target models, including multi-target regression and multi-class classification. See [Multiple Outputs](#) for more information.
 - `one_output_per_tree`: One model for each target.
 - `multi_output_tree`: Use multi-target trees.

Parameters for Non-Exact Tree Methods

- `max_cached_hist_node`, [default = 65536]

Maximum number of cached nodes for histogram. This can be used with the `hist` and the `approx` tree methods.

Added in version 2.0.0.

- For most of the cases this parameter should not be set except for growing deep trees. After 3.0, this parameter affects GPU algorithms as well.

Parameters for Categorical Feature

These parameters are only used for training with categorical data. See [Categorical Data](#) for more information.

! Note

The `exact` tree method is not supported for categorical features.

- `max_cat_to_onehot`

Added in version 1.6.0.

- A threshold for deciding whether XGBoost should use one-hot encoding based split for categorical data. When number of categories is lesser than the threshold then one-hot encoding is chosen, otherwise the categories will be partitioned into children nodes.

- `max_cat_threshold`

Added in version 1.7.0.

- Maximum number of categories considered for each split. Used only by partition-based splits for preventing over-fitting.

Additional dropout parameters for tree boosters

- `sample_type` [default= `uniform`]
 - Type of sampling algorithm.
 - `uniform` : dropped trees are selected uniformly.
 - `weighted` : dropped trees are selected in proportion to weight.
- `normalize_type` [default= `tree`]
 - Type of normalization algorithm.
 - `tree` : new trees have the same weight of each of dropped trees.
 - Weight of new trees are $1 / (k + \text{learning_rate})$.
 - Dropped trees are scaled by a factor of $k / (k + \text{learning_rate})$.
 - `forest` : new trees have the same weight of sum of dropped trees (forest).
 - Weight of new trees are $1 / (1 + \text{learning_rate})$.
 - Dropped trees are scaled by a factor of $1 / (1 + \text{learning_rate})$.
- `rate_drop` [default=0.0]
 - Dropout rate (a fraction of previous trees to drop during the dropout).
 - range: [0.0, 1.0]
- `one_drop` [default=0]
 - When this flag is enabled, at least one tree is always dropped during the dropout (allows Binomial-plus-one or epsilon-dropout from the original DART paper).
- `skip_drop` [default=0.0]
 - Probability of skipping the dropout procedure during a boosting iteration.
 - If a dropout is skipped, new trees are added in the same manner as `gbtree`.
 - Note that non-zero `skip_drop` has higher priority than `rate_drop` or `one_drop`.
 - range: [0.0, 1.0]

Parameters for Linear Booster (`booster=gblinear`)

Deprecated since version 3.3.0: `booster=gblinear` is deprecated and support will be removed in a future release.

- `lambda` [default=0, alias: `reg_lambda`]
 - L2 regularization term on weights. Increasing this value will make model more conservative. Normalised to number of training examples.
- `alpha` [default=0, alias: `reg_alpha`]
 - L1 regularization term on weights. Increasing this value will make model more conservative. Normalised to number of training examples.
- `eta` [default=0.5, alias: `learning_rate`]
 - Step size shrinkage used in update to prevent overfitting. After each boc

- range: [0,1]
- `updater` [default= `shotgun`]
 - Choice of algorithm to fit linear model
 - `shotgun` : Parallel coordinate descent algorithm based on shotgun algorithm. Uses 'hogwild' parallelism and therefore produces a nondeterministic solution on each run.
 - `coord_descent` : Ordinary coordinate descent algorithm. Also multithreaded but still produces a deterministic solution. When the `device` parameter is set to `cuda` or `gpu` , a GPU variant would be used.
- `feature_selector` [default= `cyclic`]
 - Feature selection and ordering method
 - `cyclic` : Deterministic selection by cycling through features one at a time.
 - `shuffle` : Similar to `cyclic` but with random feature shuffling prior to each update.
 - `random` : A random (with replacement) coordinate selector.
 - `greedy` : Select coordinate with the greatest gradient magnitude. It has $O(\text{num_feature}^2)$ complexity. It is fully deterministic. It allows restricting the selection to `top_k` features per group with the largest magnitude of univariate weight change, by setting the `top_k` parameter. Doing so would reduce the complexity to $O(\text{num_feature} * \text{top_k})$.
 - `thrifty` : Thrifty, approximately-greedy feature selector. Prior to cyclic updates, reorders features in descending magnitude of their univariate weight changes. This operation is multithreaded and is a linear complexity approximation of the quadratic greedy selection. It allows restricting the selection to `top_k` features per group with the largest magnitude of univariate weight change, by setting the `top_k` parameter.
- `top_k` [default=0]
 - The number of top features to select in `greedy` and `thrifty` feature selector. The value of 0 means using all the features.

Learning Task Parameters

Specify the learning task and the corresponding learning objective. The objective options are below:

- `objective` [default=reg:squarederror]
 - `reg:squarederror` : regression with squared loss.
 - `reg:squaredlogerror` : regression with squared log loss
 $\frac{1}{2} [\log(pred + 1) - \log(label + 1)]^2$. All input labels are required to be greater than -1. Also, see metric `rmsle` for possible issue with this objective.
 - `reg:logistic` : logistic regression, output probability

- `reg:absoluteerror`: Regression with L1 error. A smooth approximation is used to optimize the L1 loss.

Added in version 1.7.0.

- `reg:quantileerror`: Quantile loss, also known as `pinball loss`. A smooth approximation is used to optimize the quantile loss. See later sections for its parameter and [Prediction Intervals with Quantile and Expectile Regression](#) for a worked example.

Added in version 2.0.0.

- `reg:expectileerror`: Expectile loss (asymmetric squared error). See later sections for its parameter and properties.
- `binary:logistic`: logistic regression for binary classification, output probability
- `binary:logitraw`: logistic regression for binary classification, output score before logistic transformation
- `binary:hinge`: hinge loss for binary classification. This makes predictions of 0 or 1, rather than producing probabilities.
- `count:poisson`: Poisson regression for count data, output mean of Poisson distribution.
 - `max_delta_step` is set to 0.7 by default in Poisson regression (used to safeguard optimization)
- `survival:cox`: Cox regression for right censored survival time data (negative values are considered right censored). Note that predictions are returned on the hazard ratio scale (i.e., as $HR = \exp(\text{marginal_prediction})$ in the proportional hazard function $h(t) = h_0(t) * HR$).
- `survival:aft`: Accelerated failure time model for censored survival time data. See [Survival Analysis with Accelerated Failure Time](#) for details.
- `multi:softmax`: set XGBoost to do multiclass classification using the softmax objective, you also need to set `num_class`(number of classes)
- `multi:softprob`: same as softmax, but output a vector of `ndata * nclass`, which can be further reshaped to `ndata * nclass` matrix. The result contains predicted probability of each data point belonging to each class.
- `rank:ndcg`: Use LambdaMART to perform pair-wise ranking where [Normalized Discounted Cumulative Gain \(NDCG\)](#) is maximized. This objective supports position debiasing for click data.
- `rank:map`: Use LambdaMART to perform pair-wise ranking where [Mean Average Precision \(MAP\)](#) is maximized
- `rank:pairwise`: Use LambdaRank to perform pair-wise ranking using the *ranknet* objective.
- `reg:gamma`: gamma regression with log-link. Output is a mean of gamma distribution. It might be useful, e.g., for modeling insurance claims severity, or for any o

- `reg:tweedie`: Tweedie regression with log-link. It might be useful, e.g., for modeling total loss in insurance, or for any outcome that might be [Tweedie-distributed](#).
- `base_score`

The initial prediction score of all instances, also known as the global bias, or the intercept.

Changed in version 3.1.0: XGBoost is updated to use vector-valued intercept by default.

- The parameter is automatically estimated for selected objectives before training. To disable the estimation, specify a real number argument, e.g. `base_score = 0.5`.
- If `base_margin` is supplied, `base_score` will not be used.
- If we train the model with a sufficient number of iterations, changing this value does not offer significant benefit.

See [Intercept](#) for more information, including different use cases.

- `eval_metric` [default according to objective]
 - Evaluation metrics for validation data, a default metric will be assigned according to objective (rmse for regression, and logloss for classification, *mean average precision* for `rank:map`, etc.)
 - User can add multiple evaluation metrics. Python users: remember to pass the metrics in as list of parameters pairs instead of map, so that latter `eval_metric` won't override previous ones
 - The choices are listed below:
 - `rmse`: [root mean square error](#)
 - `rmsle`: root mean square log error: $\sqrt{\frac{1}{N} [\log(pred + 1) - \log(label + 1)]^2}$. Default metric of `reg:squaredlogerror` objective. This metric reduces errors generated by outliers in dataset. But because `log` function is employed, `rmsle` might output `nan` when prediction value is less than -1. See `reg:squaredlogerror` for other requirements.
 - `mae`: [mean absolute error](#)
 - `mape`: [mean absolute percentage error](#)
 - `mphe`: [mean Pseudo Huber error](#). Default metric of `reg:pseudohubererror` objective.
 - `expectile`: Expectile regression error (asymmetric squared error). Default metric of `reg:expectileerror` objective.
 - `logloss`: [negative log-likelihood](#)
 - `error`: Binary classification error rate. It is calculated as `#{wrong cases}/#{all cases}`. For the predictions, the evaluation will regard the instances with prediction value larger than 0.5 as positive instances, and the others as negative instances.
 - `error@t`: a different than 0.5 binary classification threshold value col

- `mlogloss` : [Multiclass logloss](#).
- `auc` : [Receiver Operating Characteristic Area under the Curve](#). Available for classification and learning-to-rank tasks.
 - When used with binary classification, the objective should be `binary:logistic` or similar functions that work on probability.
 - When used with multi-class classification, objective should be `multi:softprob` instead of `multi:softmax`, as the latter doesn't output probability. Also the AUC is calculated by 1-vs-rest with reference class weighted by class prevalence.
 - When used with multi-label classification, AUC is calculated independently for each target and averaged with the macro method.
 - When used with LTR task, the AUC is computed by comparing pairs of documents to count correctly sorted pairs. This corresponds to pairwise learning to rank. The implementation has some issues with average AUC around groups and distributed workers not being well-defined.
 - On a single machine the AUC calculation is exact. In a distributed environment the AUC is a weighted average over the AUC of training rows on each node - therefore, distributed AUC is an approximation sensitive to the distribution of data across workers. Use another metric in distributed environments if precision and reproducibility are important.
 - When input dataset contains only negative or positive samples, the output is *NaN*. The behavior is implementation defined, for instance, `scikit-learn` returns 0.5 instead.
- `aucpr` : [Area under the PR curve](#). Available for classification and learning-to-rank tasks.

After XGBoost 1.6, both of the requirements and restrictions for using `aucpr` in classification problem are similar to `auc`. For ranking task, only binary relevance label $y \in [0, 1]$ is supported. Different from `map (mean average precision)`, `aucpr` calculates the *interpolated* area under precision recall curve using continuous interpolation.

- `pre` : Precision at k . Supports only learning to rank task.
- `ndcg` : [Normalized Discounted Cumulative Gain](#)
- `map` : [Mean Average Precision](#)

The *average precision* is defined as:

$$AP@l = \frac{1}{\min(l, N)} \sum_{k=1}^l P@k \cdot I_{(k)}$$

where $I_{(k)}$ is an indicator function that equals to 1 when the document

average across all queries.

- `ndcg@n`, `map@n`, `pre@n`: n can be assigned as an integer to cut off the top positions in the lists for evaluation.
 - `ndcg-`, `map-`, `ndcg@n-`, `map@n-`: In XGBoost, the NDCG and MAP evaluate the score of a list without any positive samples as 1. By appending “-” to the evaluation metric name, we can ask XGBoost to evaluate these scores as 0 to be consistent under some conditions.
 - `poisson-nloglik`: negative log-likelihood for Poisson regression
 - `gamma-nloglik`: negative log-likelihood for gamma regression
 - `cox-nloglik`: negative partial log-likelihood for Cox proportional hazards regression
 - `gamma-deviance`: residual deviance for gamma regression
 - `tweedie-nloglik`: negative log-likelihood for Tweedie regression (at a specified value of the `tweedie_variance_power` parameter)
 - `aft-nloglik`: Negative log likelihood of Accelerated Failure Time model. See [Survival Analysis with Accelerated Failure Time](#) for details.
 - `interval-regression-accuracy`: Fraction of data points whose predicted labels fall in the interval-censored labels. Only applicable for interval-censored data. See [Survival Analysis with Accelerated Failure Time](#) for details.
- `seed` [default=0]
 - Random number seed. In the R package, if not specified, instead of defaulting to seed ‘zero’, will take a random seed through R’s own RNG engine.
 - `seed_per_iteration` [default= `false`]
 - Seed PRNG deterministically via iterator number.

Parameters for Tweedie Regression (`objective=reg:tweedie`)

- `tweedie_variance_power` [default=1.5]
 - Parameter that controls the variance of the Tweedie distribution $\text{var}(y) \sim E(y)^{\text{tweedie_variance_power}}$
 - range: (1,2)
 - Set closer to 2 to shift towards a gamma distribution
 - Set closer to 1 to shift towards a Poisson distribution.

Parameter for using Pseudo-Huber (`reg:pseudohubererror`)

- `huber_slope`: A parameter used for Pseudo-Huber loss to define the δ term.

Parameter for using Quantile Loss (`reg:quantileerror`)

- `quantile_alpha` : A scalar or an ascending list of targeted quantiles.

Added in version 2.0.0.

Parameter for using Expectile Loss (`reg:expectileerror`)

- `expectile_alpha` : A scalar or a list of targeted expectiles. Range: [0, 1]. Required for `reg:expectileerror`.

Added in version 3.3.0.

! Note

Multiple alphas must be sorted in ascending order. Unlike the quantile objective, expectile does not suffer from curve crossing. When predicting with `output_margin=True` and multiple alphas, the first margin corresponds to the smallest alpha; subsequent margins are reparameterized gaps between consecutive expectile predictions, use normal prediction to obtain the actual expectile values.

Parameter for using AFT Survival Loss (`survival:aft`) and Negative Log Likelihood of AFT metric (`aft-nloglik`)

- `aft_loss_distribution` : Probability Density Function for the AFT distribution; `normal`, `logistic`, or `extreme`.
- `aft_loss_distribution_scale` : Scaling factor for the AFT distribution. Range: (0,∞)

Parameters for learning to rank (`rank:ndcg`, `rank:map`, `rank:pairwise`)

These are parameters specific to learning to rank task. See [Learning to Rank](#) for an in-depth explanation.

- `lambdarank_pair_method` [default = `topk`]

How to construct pairs for pair-wise learning.

- `mean` : Sample `lambdarank_num_pair_per_sample` pairs for each document in the query list.
- `topk` : Focus on top-`lambdarank_num_pair_per_sample` documents. Construct $|query|$ pairs for each document at the top-`lambdarank_num_pair_per_sample` ranked by the model.
- `lambdarank_num_pair_per_sample` [range = $[1, \infty]$]

It specifies the number of pairs sampled for each document when pair method is `mean`, or the truncation level for queries when the pair method is `topk`. For example, to train with `ndcg@6`, set `lambdarank_num_pair_per_sample` to 6 and `lambdarank_pair_method` to `topk`.

- `lambdarank_normalization` [default = `true`]

Added in version 2.1.0.

Whether to normalize the leaf value by lambda gradient. This can sometimes stagnate the training progress.

Changed in version 3.0.0.

When the `mean` method is used, it's normalized by the `lambdarank_num_pair_per_sample` instead of gradient.

- `lambdarank_score_normalization` [default = `true`]

Added in version 3.0.0.

Whether to normalize the delta metric by the difference of prediction scores. This can sometimes stagnate the training progress. With pairwise ranking, we can normalize the gradient using the difference between two samples in each pair to reduce influence from the pairs that have large difference in ranking scores. This can help us regularize the model to reduce bias and prevent overfitting. Similar to other regularization techniques, this might prevent training from converging.

There was no normalization before 2.0. In 2.0 and later versions this is used by default. In 3.0, we made this an option that users can disable.

- `lambdarank_unbiased` [default = `false`]

Specify whether do we need to debias input click data.

- `lambdarank_bias_norm` [default = 2.0]

L_p normalization for position debiasing, default is L_2 . Only relevant when `lambdarank_unbiased` is set to true.

- `ndcg_exp_gain` [default = `true`]

Whether we should use exponential gain function for `NDCG`. There are two forms of gain function for `NDCG`, one is using relevance value directly while the other is using $2^{rel} - 1$ to emphasize on retrieving relevant documents. When `ndcg_exp_gain` is true (the default), relevance degree cannot be greater than 31.

Building agents? Free virtual conference, Oct 22. 122 speakers, zero vendor pitches.
Register today



Ads by
EthicalAds

Close Ad